



Graphic Era
deemed to be **University**
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

MediSense AI – A Hybrid, Self-Built k-NN Based Clinical Decision Support and Triage System

Team Name: Team #	Codeblue
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	Verma, Krish - 2510370349 2510370349@geu.ac.in 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	Kumar Kanu, shivam - 2510018717 2510018717@geu.ac.in 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	Garg, Suhani - 2510350042 2510350042@geu.ac.in 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	Sumit – 2510370550 2510370550@geu.ac.in 

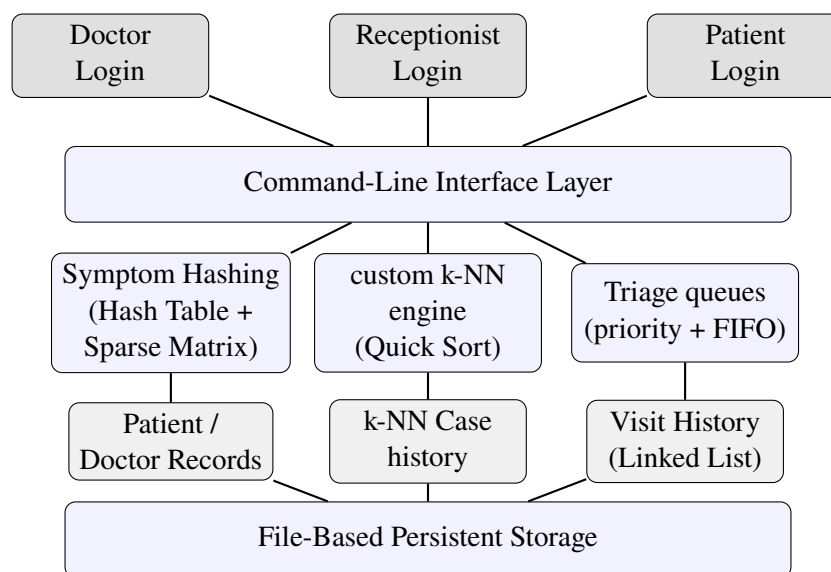
PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

MediSense AI is our C-based project that looks at how data structures and algorithms can be used in a patient intake and triage system. The user types in symptoms and vital signs. The program handles these inputs in two steps. First our nearest-neighbours (k-NN) part compares scaled values for heart rate, systolic and diastolic blood pressure, oxygen saturation and glucose with synthetic patient cases. Second another part checks the symptoms against a disease-symptom table.

The results help show a case category and suggest a specialty from the doctor list. For appointments we use a queue for routine cases and a priority queue for urgent demonstration cases. Patient, doctor, appointment and case details are meant to be stored in files. As we work on the project we're using structures, pointers, dynamic memory arrays, hashing, sorting, queues, linked lists and file handling. The code is built in a way with functions that have clear interfaces. Since the data is fake the project is for coursework and demonstration. It does not give a diagnosis or treatment advice. It should never be used of a review, by a qualified healthcare professional.

Updated Project Approach and Architecture



The workflow begins when patient details, symptoms and vital signs are entered and reviewed. The k-NN module then looks at the signs and compares them with sample cases. The symptom module checks the symptoms against its lookup table. Both results are kept apart before the program shows a case category and a possible specialty. Queue handling and saving records to files are part of the planned steps.

The features used by k-NN are heart rate, systolic blood pressure, diastolic blood pressure, oxygen saturation and glucose. These measurements have ranges so they need to be scaled using values from the training data. We are creating the application, in C. Plan to use our own data structures, standard libraries and CSV or text files. We have not decided to use a machine-learning library. The module interfaces still need to be worked out during the integration process.

Tasks Completed

Task Completed	Team Member
Worked on combining the k-NN vital-sign result with symptom matching so the program can show a case category and suggest a relevant specialist.	Krish Verma
Added role-based login options for doctors, patients, and receptionists, along with input for registering a new patient.	Shivam Kumar Kanu
Organized the patient and doctor datasets and worked on updating records when new-patient details are entered.	Sumit
Added appointment booking to the Patient and Receptionist sections and checked the related input and booking steps, along with report viewing.	Suhani Garg

Challenges/Roadblocks

One limitation we must remember is the data itself. Our patient cases are made up. They help us test the programs logic. They cannot show how the program would work with real medical records. We treat MediSense AI as a learning project not as a tool for making decisions. Any real case would need assessment by a health professional. A second issue is that vital signs do not share the units or numerical range. Before comparing cases we need to scale the values according to the training dataset and use the same settings for new entries.

We also have two ways to analyse an entry. K-NN looks for cases with similar vital signs; symptom matching checks the entered symptoms against the disease-symptom table. Showing the two outputs separately will make it easier to understand what each method returned. We will combine them only if their categories can be compared meaningfully. Integration is another task to watch: modules may use field names, data formats or function arguments. Our plan is to agree on shared structures check user input and test the modules individually before joining them. Once the integrated program is ready we still need to record tests, for urgent-demo, incomplete and invalid inputs.

Tasks Pending

Task Pending	Team Member (to complete the task)
Implement emergency-case handling and doctor scheduling.	Suhani Garg
Enable patients to view their reports and doctors to update patient reports.	Sumit
Develop and integrate the user interface for Doctor, Patient, and Receptionist features.	Shivam Kumar kanu and Krish Verma

Project Outcome/Deliverables

Our goal is to finish MediSense AI as a menu-driven C program that brings together the data structures covered in our coursework. A user will be able to enter symptoms and vital signs; the k-NN module will compare the vital signs with synthetic sample cases, and the symptom module will check the reported symptoms against its table. The program is intended to show a case category and use it to suggest a suitable specialty from the doctor list.

Other planned parts are regular and priority queues for appointment handling, file-based patient, doctor, and case records, and input checks. Building these parts gives us a chance to apply structures, pointers, dynamic memory allocation, arrays, hashing, sorting, queues, linked lists, and file handling in one project.

We plan to submit the C source code, synthetic patient and doctor datasets, test cases with their results, the project report, and a presentation of the workflow. The program is for learning only; it cannot provide a medical diagnosis or treatment advice.

Progress Overview

We have defined the purpose of MediSense AI, the people who will use MediSense AI and how the different parts of MediSense AI are expected to work together. The design of MediSense AI includes patient-data entry k-NN analysis of signs, symptom matching, specialty suggestions, queue management and file storage. We have linked these planned features of MediSense AI to topics from our Data Structures and OOP courses. We have also prepared one hundred patient cases and a list of doctors with relevant specialties for MediSense AI. The Phase-I presentation says that work on k-NN analysis and symptom matching has started, but it does not confirm that both modules of MediSense AI are complete or connected.

The complete C application, file handling, triage flow and systematic algorithm testing for MediSense AI still need work. Our next steps for MediSense AI are to settle a data format integrate the modules of MediSense AI check valid and invalid inputs and record the test results, for MediSense AI. We will also keep the repository address, branch and important commit details of MediSense AI in the report.

Codebase Information

Language: C. Current features include patient-data entry, role-based login, k-NN and symptom-based results, specialty suggestions, and appointment booking.

Repository: <https://github.com/19Krishv/Medisense-Ai>

Branch: main

Current history: 14 commits. Visible commits include a1cfba2 – Test History, history.h module, Symptom Hash, project report updates; k-NN prediction code; Doctor, Patient, and Receptionist login portals; new-patient detail input; and updating patient records.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Patient input and role access	Checked	Workflow checked by Suhani.
Appointment booking	Checked	Patient and Receptionist booking workflow checked by Suhani.
Prediction and specialist suggestion	Checked	Test k-NN and symptom-based results using sample cases.
Patient reports, doctor updates and emergency cases priority	Pending	Sumit is completing report viewing and doctor update features.

Deliverables Progress

Completed: Role-based access, new-patient input, k-NN and symptom-based prediction with specialist suggestions, and appointment booking through Patient and Receptionist sections.

In progress: Emergency-case handling and doctor scheduling.

Pending: Patient report viewing and doctor updates to patient reports.